2.3 Cálculo eficiente de métricas con TorchMetrics

Parte 1: ¿Cómo funciona TorchMetrics?

TorchMetrics

Gratis y de código abierto.

https://torchmetrics.readthedocs.io

Instalar TorchMetrics

```
pip install torchmetrics
```

Antes


```python
def compute_accuracy(model, dataloader):

    model = model.eval()

    correct = 0.0
    total_examples = 0

    for idx, (features, labels) in enumerate(dataloader):

        with torch.no_grad():
            logits = model(features)

        predictions = torch.argmax(logits, dim=1)
        compare = labels == predictions
        correct += torch.sum(compare)
        total_examples += len(compare)

    return correct / total_examples


val_acc = compute_accuracy(model, val_loader)
val_acc
```

```
tensor(0.9646)
```

```python
def compute_accuracy(model, dataloader):

    model = model.eval()

    correct = 0.0
    total_examples = 0

    for idx, (features, labels) in enumerate(dataloader):

        with torch.no_grad():
            logits = model(features)

        predictions = torch.argmax(logits, dim=1)
        compare = labels == predictions
        correct += torch.sum(compare)
        total_examples += len(compare)

    return correct / total_examples


val_acc = compute_accuracy(model, val_loader)
val_acc
```

Para cada batch

Calcular etiquetas

Contar etiquetas correctas

```
tensor(0.9646)
```

```python
def compute_accuracy(model, dataloader):

    model = model.eval()

    correct = 0.0
    total_examples = 0

    for idx, (features, labels) in enumerate(dataloader):

        with torch.no_grad():
            logits = model(features)

        predictions = torch.argmax(logits, dim=1)
        compare = labels == predictions
        correct += torch.sum(compare)
        total_examples += len(compare)

    return correct / total_examples


val_acc = compute_accuracy(model, val_loader)
val_acc
```

Calcular el promedio

```
tensor(0.9646)
```

¿Por qué incrementalmente?

Limitación de memoria al cargar.
No se puede cargar todo el conjunto de datos en memoria al mismo tiempo para datasets grandes

TorchMetrics sigue un concepto similar de calcular la exactitud (y otras métricas) de forma incremental.

```python
def compute_accuracy(model, dataloader):

    model = model.eval()

    correct = 0.0
    total_examples = 0

    for idx, (features, labels) in enumerate(dataloader):

        with torch.no_grad():
            logits = model(features)

        predictions = torch.argmax(logits, dim=1)
        compare = labels == predictions
        correct += torch.sum(compare)
        total_examples += len(compare)

    return correct / total_examples


val_acc = compute_accuracy(model, val_loader)
val_acc
```

```
tensor(0.9646)
```

```python
import torchmetrics

def compute_accuracy(model, dataloader):

    model = model.eval()

    acc = torchmetrics.Accuracy(task="multiclass", num_classes=10)

    for idx, (features, labels) in enumerate(dataloader):

        with torch.no_grad():
            logits = model(features)

        predictions = torch.argmax(logits, dim=1)
        acc(predictions, labels)

    return acc


val_acc = compute_accuracy(model, val_loader)
val_acc.compute()
```

```
tensor(0.9646)
```

Más allá de la exactitud de clasificación

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

TorchMetrics incluye más de 90 implementaciones de métricas para PyTorch

Siguiente:
Agregar el cálculo de exactitud de TorchMetrics a nuestro modelo

2.4 Haciendo el código reproducible

Parte 1: Fuentes de aleatoriedad

Generar un número aleatorio en Python

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```
[1]: import random

[2]: random.random()

[2]: 0.5216761588732212
```

(Esto se verá diferente en tu computador)

```
[1]: import random

[2]: random.random()

[2]: 0.5216761588732212

[3]: random.random()

[3]: 0.1550530569802705
```

(Esto se verá diferente en tu computador)

Sembramos el generador de números aleatorios

Configurar una semilla de número aleatorio

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```
[4]: random.seed(1)
     random.random()
```

```
[4]: 0.13436424411240122
```

(Esto se verá igual en tu computador)

Ahora llamemos Random() repetidamente

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

```
[1]: import random

[2]: random.random()

[2]: 0.5216761588732212
```

(Esto se verá igual en tu computador)

La secuencia de ejecución se vuelve reproducible cuando fijamos la semilla del generador de números aleatorios

```
[4]: random.seed(1)
     random.random()

[4]: 0.13436424411240122

[5]: random.random()

[5]: 0.8474337369372327

[6]: random.seed(1)
     random.random()

[6]: 0.13436424411240122

[7]: random.random()

[7]: 0.8474337369372327
```

El mismo concepto aplica a PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Algunas fuentes comunes de aleatoriedad son:

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

La mayoría de los scripts de entrenamiento en deep learning no son reproducibles entre distintos entornos. Sin embargo, los modelos entrenados (usualmente) se comportan de forma determinista

Siguiente: Controlar fuentes comunes de aleatoriedad

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

2.4 Haciendo el código reproducible

Parte 2: Controlando las fuentes de aleatoriedad

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

[1]:
```python
import torch


linear = torch.nn.Linear(in_features=5, out_features=10)
```

```
[1]: import torch


     linear = torch.nn.Linear(in_features=5, out_features=10)
     linear.weight[0]
```

```
[1]: tensor([-0.1521,  0.2737, -0.3880, -0.3918, -0.0185],
            grad_fn=<SelectBackward0>)
```

(Esto se verá diferente en tu computador)

```
[1]: import torch


     linear = torch.nn.Linear(in_features=5, out_features=10)
     linear.weight[0]
```

```
[1]: tensor([-0.1521,  0.2737, -0.3880, -0.3918, -0.0185],
            grad_fn=<SelectBackward0>)
```

```
[2]: linear = torch.nn.Linear(in_features=5, out_features=10)
     linear.weight[0]
```

```
[2]: tensor([-0.2645, -0.3156,  0.2655, -0.2972,  0.0903],
            grad_fn=<SelectBackward0>)
```

(Esto se verá diferente en tu computador)

```python
torch.manual_seed(123)

linear = torch.nn.Linear(in_features=5, out_features=10)
linear.weight[0]
```

```
tensor([-0.1824,  0.0148, -0.2221,  0.1687, -0.3811],
       grad_fn=<SelectBackward0>)
```

(Esto se verá igual en tu computador)

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

```python
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


train_dataset = datasets.MNIST(
    root="./mnist", train=True, transform=transforms.ToTensor(), download=True
)

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=10,
    shuffle=True,
)

for images, labels in train_loader:
    pass

print("Labels of last batch", labels)
```

```
Labels of last batch tensor([4, 2, 8, 6, 6, 2, 6, 7, 7, 4])
```

```python
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


train_dataset = datasets.MNIST(
    root="./mnist", train=True, transform=transforms.ToTensor(), download=True
)

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=10,
    shuffle=True,
)

for images, labels in train_loader:
    pass

print("Labels of last batch", labels)
```

```
Labels of last batch tensor([4, 2, 8, 6, 6, 2, 6, 7, 7, 4])
```

```python
for images, labels in train_loader:
    pass

print("Labels of last batch", labels)
```

```
Labels of last batch tensor([9, 6, 6, 1, 0, 3, 0, 1, 7, 8])
```

```python
torch.manual_seed(123)

for images, labels in train_loader:
    pass

print("Labels of last batch", labels)
```

```
Labels of last batch tensor([1, 8, 1, 8, 8, 7, 2, 5, 4, 1])
```

(Esto se verá igual en tu computador)

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

```python
trainer = L.Trainer(
    max_epochs=2,
    accelerator="auto",
    devices="auto",
    deterministic=True
)
```

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

1. Inicialización de pesos del modelo
2. Mezcla y aumento del dataset
3. Algoritmos no deterministas
4. Hardware y drivers (GPU)

Si queremos garantizar un comportamiento reproducible,

- se requiere usar el mismo hardware y driver de GPUo

- usar siempre CPUs (compensación: será más lento)

Las ejecuciones en CPU usualmente son reproducibles

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```
trainer = L.Trainer(
    max_epochs=2,
    accelerator="cpu",
    deterministic=True
)
```

Siguiente:
Hacer reproducible nuestro script de entrenamiento en Lightning

2.5 Organizar tus cargadores de datos con Data Modules

Parte 1: Data Modules como una capa opcional de organización

Un DataModule es una clase compartible y reutilizable que encapsula la lógica de (pre)procesamiento de datos y de carga de datos

```python
class MNISTDataModule(LightningDataModule):
    def __init__(self, data_dir="path/to/dir", batch_size=32):
        super().__init__()
        self.data_dir = data_dir
        self.batch_size = batch_size

    def prepare_data(self):
        # download
        MNIST(self.data_dir, train=True, download=True)
        MNIST(self.data_dir, train=False, download=True)

    def setup(self, stage: str):
        self.mnist_test = MNIST(self.data_dir, train=False)
        self.mnist_predict = MNIST(self.data_dir, train=False)
        mnist_full = MNIST(self.data_dir, train=True)
        self.mnist_train, self.mnist_val = random_split(mnist_full, [55000, 5000])

    def train_dataloader(self):
        return DataLoader(self.mnist_train, batch_size=self.batch_size)

    def val_dataloader(self):
        return DataLoader(self.mnist_val, batch_size=self.batch_size)

    def test_dataloader(self):
        return DataLoader(self.mnist_test, batch_size=self.batch_size)

    def predict_dataloader(self):
        return DataLoader(self.mnist_predict, batch_size=self.batch_size)

    def teardown(self, stage: str):
        # Used to clean-up when the run is finished
        ...
```

La lógica de descarga del dataset va aquí(evita duplicación en configuraciones de entrenamiento multi-GPU

```python
class MNISTDataModule(LightningDataModule):
    def __init__(self, data_dir="path/to/dir", batch_size=32):
        super().__init__()
        self.data_dir = data_dir
        self.batch_size = batch_size

    def prepare_data(self):
        # download
        MNIST(self.data_dir, train=True, download=True)
        MNIST(self.data_dir, train=False, download=True)

    def setup(self, stage: str):
        self.mnist_test = MNIST(self.data_dir, train=False)
        self.mnist_predict = MNIST(self.data_dir, train=False)
        mnist_full = MNIST(self.data_dir, train=True)
        self.mnist_train, self.mnist_val = random_split(mnist_full, [55000, 5000])

    def train_dataloader(self):
        return DataLoader(self.mnist_train, batch_size=self.batch_size)

    def val_dataloader(self):
        return DataLoader(self.mnist_val, batch_size=self.batch_size)

    def test_dataloader(self):
        return DataLoader(self.mnist_test, batch_size=self.batch_size)

    def predict_dataloader(self):
        return DataLoader(self.mnist_predict, batch_size=self.batch_size)

    def teardown(self, stage: str):
        # Used to clean-up when the run is finished
        ...
```

Los pasos que pueden/necesitan ejecutarse en cada GPU van aquí

```python
class MNISTDataModule(LightningDataModule):
    def __init__(self, data_dir="path/to/dir", batch_size=32):
        super().__init__()
        self.data_dir = data_dir
        self.batch_size = batch_size

    def prepare_data(self):
        # download
        MNIST(self.data_dir, train=True, download=True)
        MNIST(self.data_dir, train=False, download=True)

    def setup(self, stage: str):
        self.mnist_test = MNIST(self.data_dir, train=False)
        self.mnist_predict = MNIST(self.data_dir, train=False)
        mnist_full = MNIST(self.data_dir, train=True)
        self.mnist_train, self.mnist_val = random_split(mnist_full, [55000, 5000])

    def train_dataloader(self):
        return DataLoader(self.mnist_train, batch_size=self.batch_size)

    def val_dataloader(self):
        return DataLoader(self.mnist_val, batch_size=self.batch_size)

    def test_dataloader(self):
        return DataLoader(self.mnist_test, batch_size=self.batch_size)

    def predict_dataloader(self):
        return DataLoader(self.mnist_predict, batch_size=self.batch_size)

    def teardown(self, stage: str):
        # Used to clean-up when the run is finished
        ...
```

Configura los data loaders como de costumbre

```python
class MNISTDataModule(LightningDataModule):
    def __init__(self, data_dir="path/to/dir", batch_size=32):
        super().__init__()
        self.data_dir = data_dir
        self.batch_size = batch_size

    def prepare_data(self):
        # download
        MNIST(self.data_dir, train=True, download=True)
        MNIST(self.data_dir, train=False, download=True)

    def setup(self, stage: str):
        self.mnist_test = MNIST(self.data_dir, train=False)
        self.mnist_predict = MNIST(self.data_dir, train=False)
        mnist_full = MNIST(self.data_dir, train=True)
        self.mnist_train, self.mnist_val = random_split(mnist_full, [55000, 5000])

    def train_dataloader(self):
        return DataLoader(self.mnist_train, batch_size=self.batch_size)

    def val_dataloader(self):
        return DataLoader(self.mnist_val, batch_size=self.batch_size)

    def test_dataloader(self):
        return DataLoader(self.mnist_test, batch_size=self.batch_size)

    def predict_dataloader(self):
        return DataLoader(self.mnist_predict, batch_size=self.batch_size)

    def teardown(self, stage: str):
        # Used to clean-up when the run is finished
        ...
```

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Estos son usados por …

trainer.fit()

trainer.fit()
trainer.validate()

trainer.test()

trainer.predict()

Una vez que definimos el DataModule, su uso es directo:

```
dm = MNISTDataModule()
model = Model()

trainer.fit(model, datamodule=dm)
trainer.test(datamodule=dm)
trainer.validate(datamodule=dm)
trainer.predict(datamodule=dm)
```

Siguiente:
Incorporar Data Modules en nuestro script de entrenamiento con Lightning

2.6 Los beneficios del registro

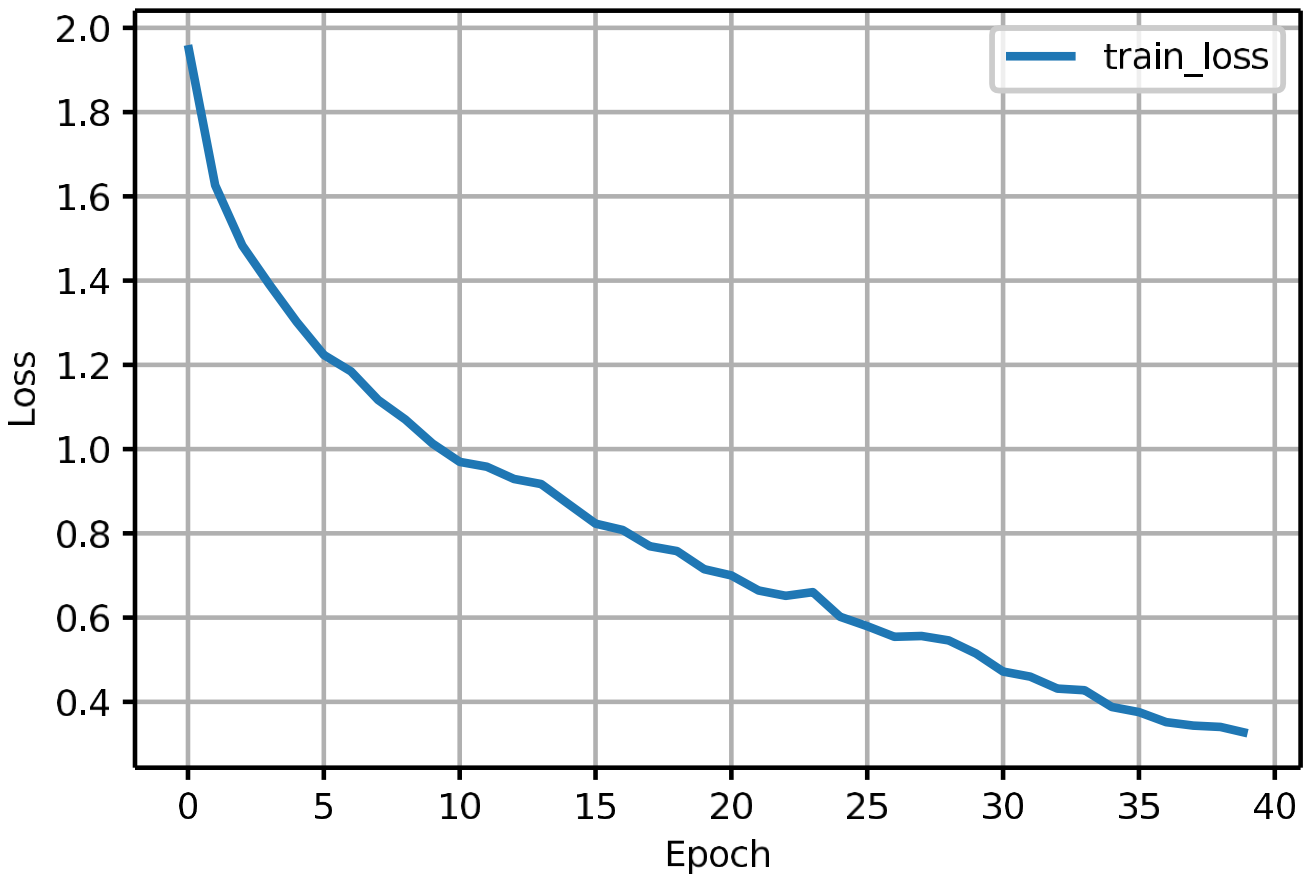
Parte 1: Seguimiento del progreso del entrenamiento

¿Por qué registrar (logging)?

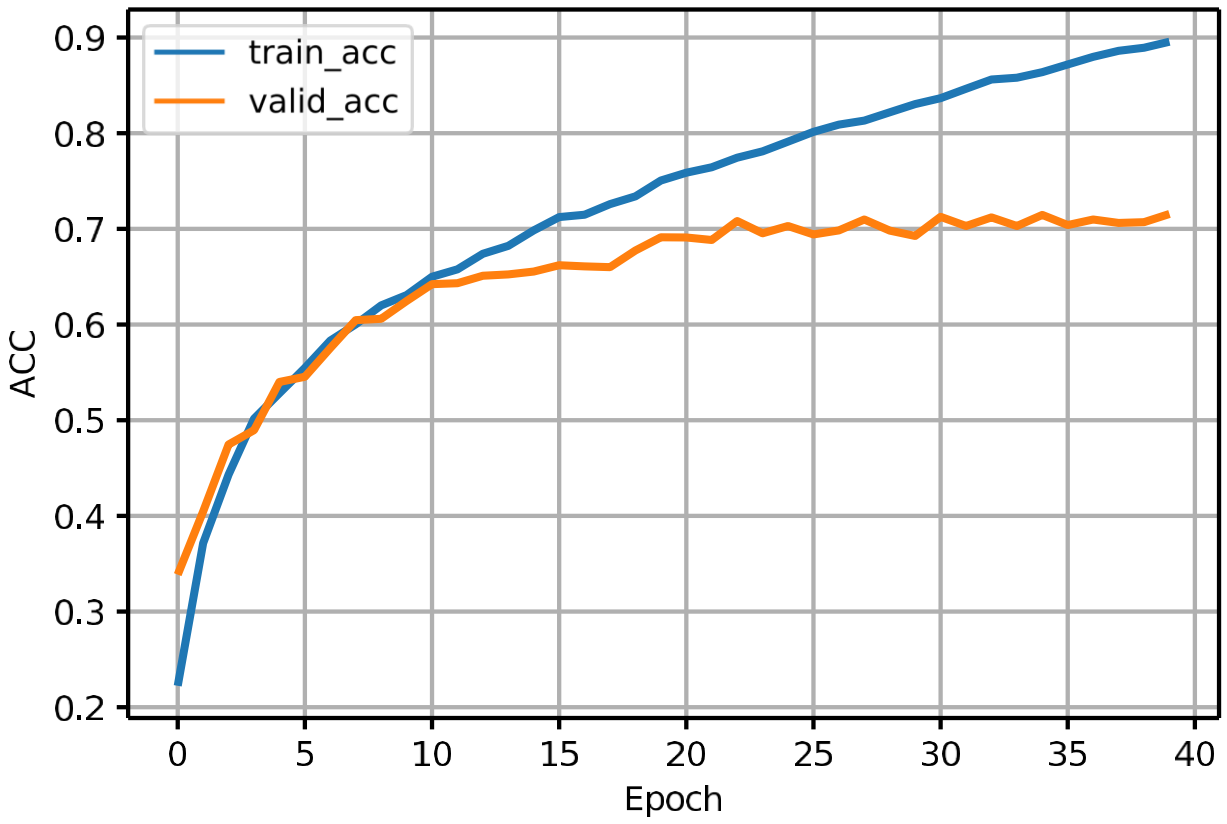
- Monitorear el progreso del entrenamiento
- Depurar un modelo
- Mejorar un modelo
- Mantener registro

¿Cosas básicas a las que debemos prestar atención?

¿La pérdida de entrenamiento disminuye?



¿El modelo está sobreajustando?



Guardar hiperparámetros

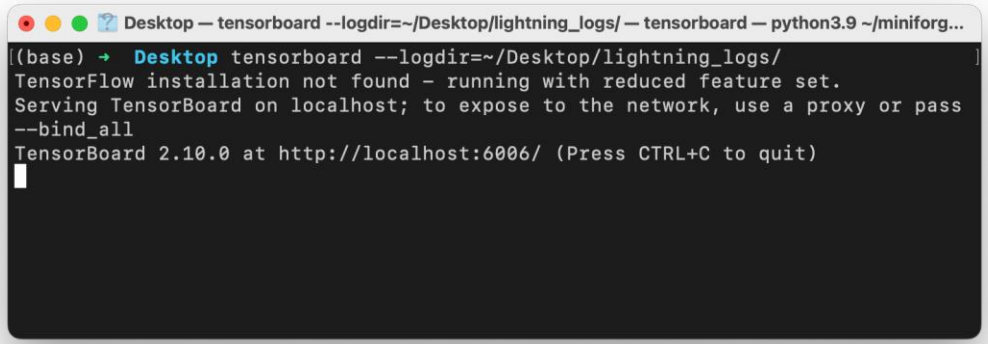
UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

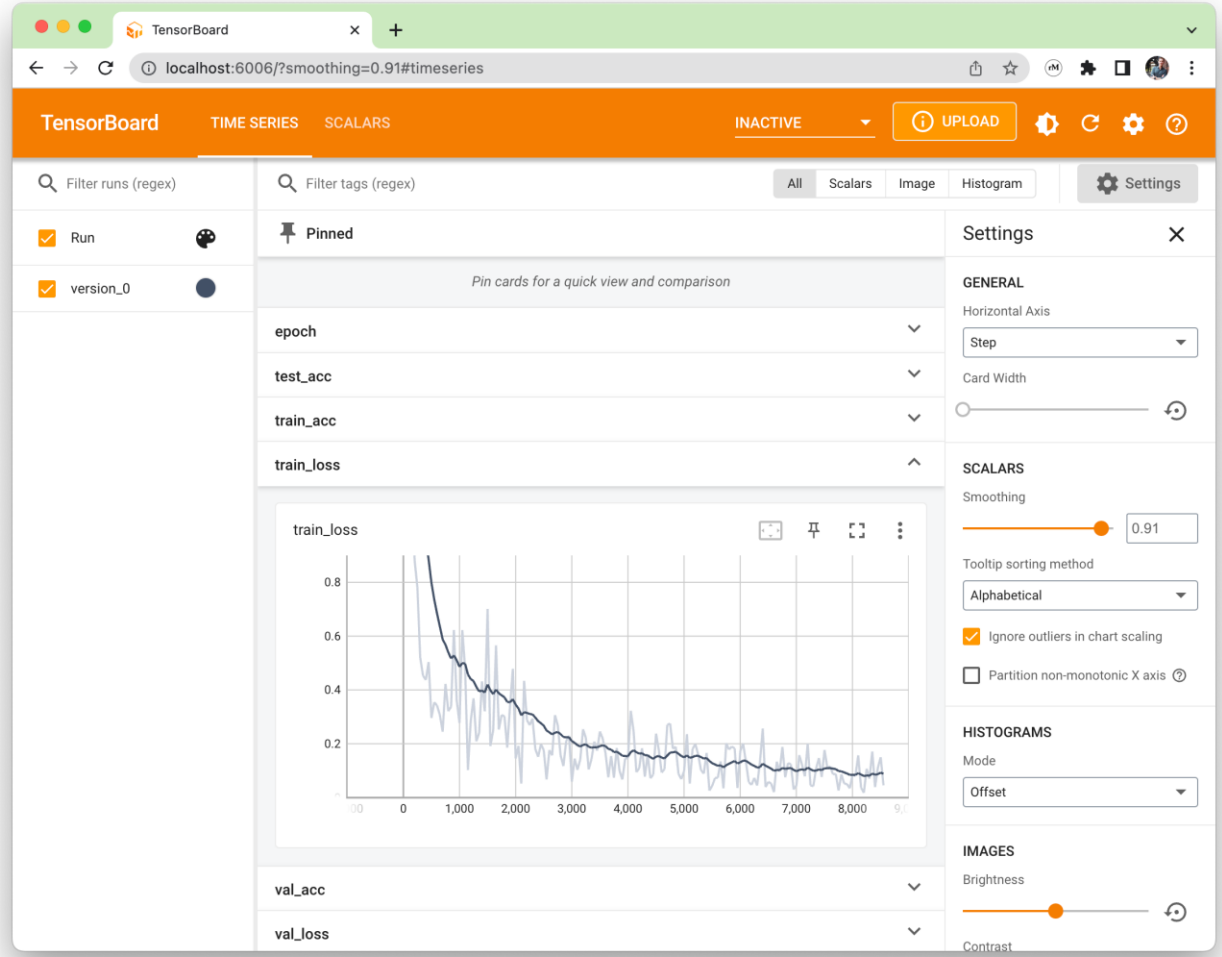
```python
class LightningModel(L.LightningModule):
    def __init__(self, model, learning_rate):
        super().__init__()

        self.learning_rate = learning_rate
        self.model = model

        # Save settings and hyperparameters to the log directory
        # but skip the model parameters
        self.save_hyperparameters(ignore=["model"])

        self.train_acc = torchmetrics.Accuracy()
        self.val_acc = torchmetrics.Accuracy()
        self.test_acc = torchmetrics.Accuracy()

    def forward(self, x):
        return self.model(x)

    ...
```

Actualmente, los siguientes loggers son compatibles:configuración predeterminada

`CometLogger`	Track your parameters, metrics, source code and more using Comet.
`CSVLogger`	Log to local file system in yaml and CSV format.
`MLFlowLogger`	Log using MLflow.
`NeptuneLogger`	Log using Neptune.
`TensorBoardLogger`	Log to local file system in TensorBoard format.
`WandbLogger`	Log using Weights and Biases.

configuración predeterminada


Desktop — tensorboard --logdir=~/Desktop/lightning_logs/ — tensorboard — python3.9 ~/miniforg...
(base) → Desktop tensorboard --logdir=~/Desktop/lightning_logs/
TensorFlow installation not found - running with reduced feature set.
Serving TensorBoard on localhost; to expose to the network, use a proxy or pass --bind_all
TensorBoard 2.10.0 at http://localhost:6006/ (Press CTRL+C to quit)


TensorBoard
localhost:6006/?smoothing=0.91#timeseries
TIME SERIES
SCALARS
INACTIVE
UPLOAD
Filter runs (regex)
Filter tags (regex)
All
Scalars
Image
Histogram
Settings
Run
version_0
Pinned
Pin cards for a quick view and comparison
epoch
test_acc
train_acc
train_loss
train_loss
0.8
0.6
0.4
0.2
0
1,000
2,000
3,000
4,000
5,000
6,000
7,000
8,000
val_acc
val_loss
Settings
GENERAL
Horizontal Axis
Step
Card Width
SCALARS
Smoothing
0.91
Tooltip sorting method
Alphabetical
Ignore outliers in chart scaling
Partition non-monotonic X axis
HISTOGRAMS
Mode
Offset
IMAGES
Brightness
Contrast

Siguiente:
Revisar los registros de nuestro modelo en Lightning

2.7 Evaluando y usando modelos en datos nuevos

Parte 1: Pasos fundamentales de inspección del modelo

Guardar hiperparámetros

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
class LightningModel(L.LightningModule):
    def __init__(self, model, learning_rate):
        super().__init__()

        self.learning_rate = learning_rate
        self.model = model

        # Save settings and hyperparameters to the log directory
        # but skip the model parameters
        self.save_hyperparameters(ignore=["model"])

        self.train_acc = torchmetrics.Accuracy()
        self.val_acc = torchmetrics.Accuracy()
        self.test_acc = torchmetrics.Accuracy()

    def forward(self, x):
        return self.model(x)

    ...
```

Guardar y cargar un checkpoint del modelo

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
pytorch_model = PyTorchMLP(...)
lightning_model = LightningModel(model=pytorch_model, ...)

trainer = L.Trainer(...)
trainer.fit(model=lightning_model, datamodule=dm)
```

```python
trainer.save_checkpoint("example.ckpt")
LightningModel.load_from_checkpoint(checkpoint_path="example.ckpt", model=pytorch_model)
```

Guardar y cargar un checkpoint del modelo

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```
pytorch_model = PyTorchMLP(...)
lightning_model = LightningModel(model=pytorch_model, ...)

trainer = L.Trainer(...)
trainer.fit(model=lightning_model, datamodule=dm)
```

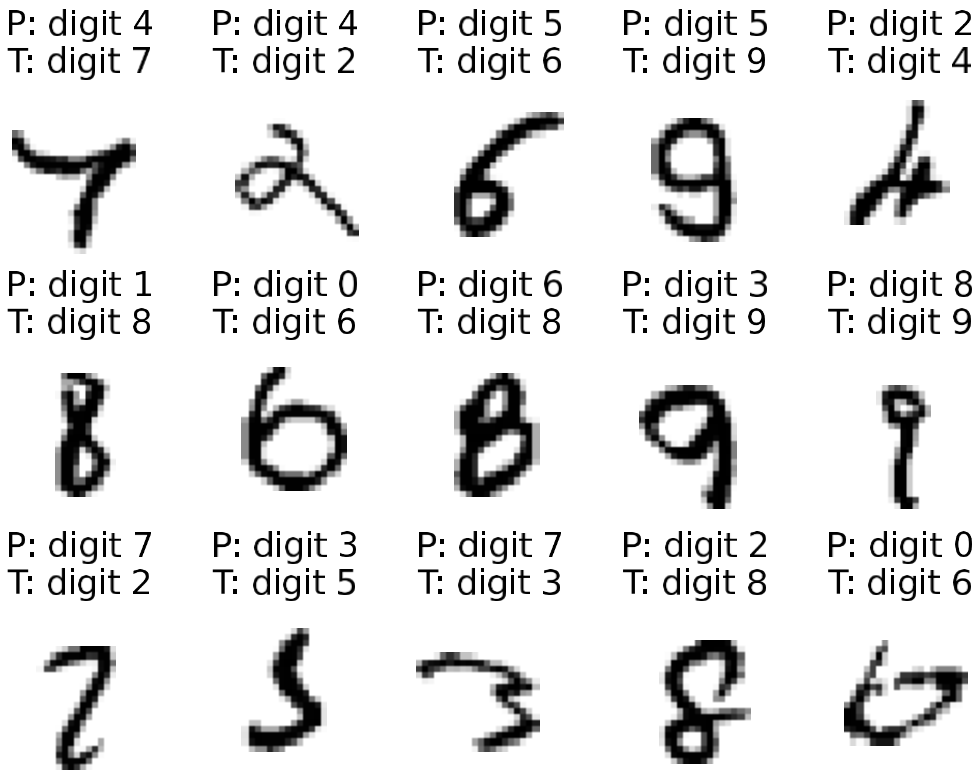
Previamente excluido de self.save_hyperparameters()

```
trainer.save_checkpoint("example.ckpt")
LightningModel.load_from_checkpoint(checkpoint_path="example.ckpt", model=pytorch_model)
```

Usar el modelo en datos nuevos


```python
lightning_model.eval()

with torch.inference_mode():
    logits = lightning_model(new_data)
```

Examinar casos de falla



Matrices de confusión

true label	digit 0	digit 1	digit 2	digit 3	digit 4	digit 5	digit 6	digit 7	digit 8	digit 9
digit 0	963	0	2	2	0	6	3	2	2	0
digit 1	0	1122	2	3	0	0	3	1	4	0
digit 2	5	1	1006	5	2	0	4	5	4	0
digit 3	1	0	1	991	0	7	0	6	2	2
digit 4	2	0	2	1	950	3	9	2	0	13
digit 5	2	0	0	11	1	869	4	1	3	1
digit 6	5	2	1	1	3	7	938	0	1	0
digit 7	2	7	12	7	1	0	1	990	5	3
digit 8	8	3	3	11	3	23	6	4	912	1
digit 9	5	5	0	13	9	9	0	9	7	952

Siguiente:
Evaluemos nuestro clasificador de dígitos anteriormente

5.8 Agregar funcionalidad con callbacks

¿Qué es un callback?

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Un programa/función autónomo para extender funcionalidad existente

Se conecta (hook) dentro de tu código

¿Qué puedes hacer con los callbacks?

- Actualizar tasas de aprendizaje
- Visualizar gradientes
- Enviar correos durante el entrenamiento (o cuando termine)
- …
- Solo estás limitado por tu imaginación

Revisando un callback preconstruido de checkpointing

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
from lightning.pytorch.callbacks import ModelCheckpoint

lightning_model = LightningModel(pytorch_model, learning_rate=...)

checkpoint_callback = ModelCheckpoint(
        save_top_k=1, mode="max", monitor="valid_acc"
)  # save top 1 model



trainer = L.Trainer(
    max_epochs=NUM_EPOCHS,
    callbacks=[callbacks],
    ...
)
```

Construyamos nuestro callback personalizado

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
from pytorch_lightning.callbacks import Callback
import time


class CustomTimingCallback(Callback):
    def on_train_start(self, trainer, lightning_module):
        self.start = time.time()
        print("Training is starting.")

    def on_train_end(self, trainer, lightning_module):
        self.end = time.time()
        total_minutes = (self.end - self.start) / 60
        print(f"Training has finished. It took {total_minutes} min.")
```

Construyamos nuestro callback personalizado

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
from pytorch_lightning.callbacks import Callback
import time


class CustomTimingCallback(Callback):
    def on_train_start(self, trainer, lightning_module):
        self.start = time.time()
        print("Training is starting.")

    def on_train_end(self, trainer, lightning_module):
        self.end = time.time()
        total_minutes = (self.end - self.start) / 60
        print(f"Training has finished. It took {total_minutes} min.")


callbacks=[CustomTimingCallback()]

trainer = L.Trainer(
    max_epochs=5,
    callbacks=callbacks,
)

trainer.fit(model=lightning_model, datamodule=dm)
```

Construyamos nuestro callback personalizado

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

```python
from pytorch_lightning.callbacks import Callback
import time


class CustomTimingCallback(Callback):
    def on_train_start(self, trainer, lightning_module):
        self.start = time.time()
        print("Training is starting.")

    def on_train_end(self, trainer, lightning_module):
        self.end = time.time()
        total_minutes = (self.end - self.start) / 60
        print(f"Training has finished. It took {total_minutes} min.")


callbacks=[CustomTimingCallback()]

trainer = L.Trainer(
    max_epochs=5,
    callbacks=callbacks,
)

trainer.fit(model=lightning_model, datamodule=dm)
```

```
GPU available: True (mps), used: False
TPU available: False, using: 0 TPU cores
IPU available: False, using: 0 IPUs
HPU available: False, using: 0 HPUs

  | Name      | Type       | Params
----------------------------------------
0 | model     | PyTorchMLP | 40.8 K
1 | train_acc | Accuracy   | 0
2 | val_acc   | Accuracy   | 0
3 | test_acc  | Accuracy   | 0
----------------------------------------
40.8 K    Trainable params
0         Non-trainable params
40.8 K    Total params
0.163     Total estimated model params size (MB)
Training is starting.
Epoch 4: 100% ■■■■■■■■■■ 938/938 [00:03<00:00, 235.17it/s, loss=0.155, v_num=1, val_loss=0.171, val_acc=0.953, train_acc=0.954]
`Trainer.fit` stopped: `max_epochs=5` reached.
Training has finished. It took 0.33937496741612755 min.
```

Callback hooks

Hay muchísimos hooks …

Docs > accelerators > ModelHooks

Edit on GitHub

on_train_epoch_start ( ) [SOURCE]

Called in the training loop at the very beginning of the epoch.

Return type

None

on_train_start ( ) [SOURCE]

Called at the beginning of training after sanity check.

Return type

None

on_validation_batch_end (*outputs* , *batch* , *batch_idx* , *dataloader_idx*) [SOURCE]

Called in the validation loop after the batch.

Parameters

- **outputs** (Union [Tensor , Dict [str , Any], None]) – The outputs of validation_step_end(validation_step(x))
- **batch** (Any) – The batched data as it is returned by the validation DataLoader.
- **batch_idx** (int) – the index of the batch
- **dataloader_idx** (int) – the index of the dataloader

Return type

None

on_validation_batch_start (*batch* , *batch_idx* , *dataloader_idx*) [SOURCE]

Called in the validation loop before anything happens for that batch.

Parameters

Siguiente: Consejos y trucos esenciales de deep learning (y callbacks en práctica)